PhysicalAI-ROCm

Dual-Domain Physical AI Demonstrations
on AMD Radeon GPU and ROCm

AMD AI DevMaster Hackathon
Track 3 - Physical AI Challenge

Registered participant: daotianyi
GitHub: daotianyi
Submission type: Individual

TECHNICAL REPORT | AUGUST 2026

Executive Summary

PhysicalAI-ROCm is a compact reproducibility package containing two simulation-based Physical AI demonstrations on one AMD ROCm stack. The HighwayEnv demonstration trains a DQN policy for high-level driving decisions and evaluates it against a seeded random baseline. The Genesis demonstration uses inverse kinematics and joint PD control to make a Franka Panda manipulator follow a moving three-dimensional target while rendering an off-screen video.

The engineering goal is transparent evidence: exact commands, fixed seeds, machine-readable metrics, GPU identity, framework versions, limitations, and outputs that evaluators can regenerate.

Submission Snapshot

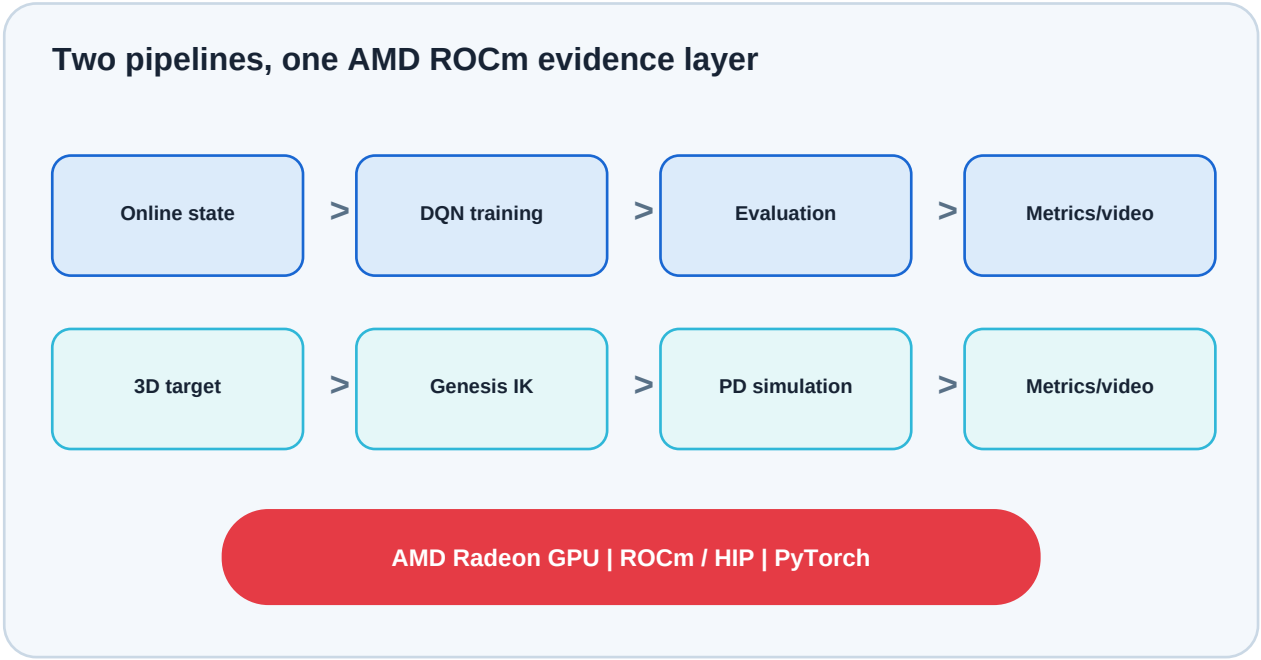
Component	Method	Primary evidence
Highway driving	DQN vs random policy in highway-fast-v0	20 seeded episodes, reward, collision rate, latency
Robot manipulation	Genesis IK and joint PD target following	300 frames, tracking error, IK time, rendered video
Compute stack	AMD Radeon GPU, ROCm/HIP, PyTorch	Runtime checks and JSON version fields

Scope

The project demonstrates simulation, reinforcement learning, policy inference, robot motion control, rendering, and evaluation. It does not claim production autonomous driving, physical grasping, or real-robot deployment.

1. Target Application and System Design

The target audience is engineers and students who need a fast, auditable path from an AMD Radeon environment to working Physical AI examples. The two tasks cover complementary patterns: learning a decision policy from online environment interaction and controlling a robot from a model-based kinematic objective.



Shared engineering layer

- AMD Radeon GPU execution through ROCm/HIP.
- Headless operation for Radeon Cloud, SSH, and containers.
- Deterministic seeds and explicit evaluation settings.
- JSON metrics and MP4 video outputs.
- A single README with installation, commands, expected results, and limitations.

2. Data and Evaluation Protocol

No external training dataset is used. Both demonstrations generate state inside simulation, which avoids licensing and download dependencies while making the data-generation process fully inspectable.

HighwayEnv

Training transitions are generated online from **highway-fast-v0**. The observation is a compact kinematic representation. The discrete action space contains lane left, idle, lane right, faster, and slower. The submitted comparison evaluates both random and learned policies over 20 episodes with seeds 100 through 119.

Genesis Franka

No policy is trained. A procedural sinusoidal function generates a 300-frame three-dimensional figure-eight trajectory using seed 42. Each target is shifted 8 cm upward to define the desired hand position. Euclidean tracking error is computed from the measured and desired hand positions.

Fairness and repeatability

The same HighwayEnv configuration and evaluation seeds are applied to both policies. Generated metrics preserve episode-level results. The Genesis result records simulation time step, physics steps per frame, target frame count, seed, framework versions, and GPU identity.

Important interpretation

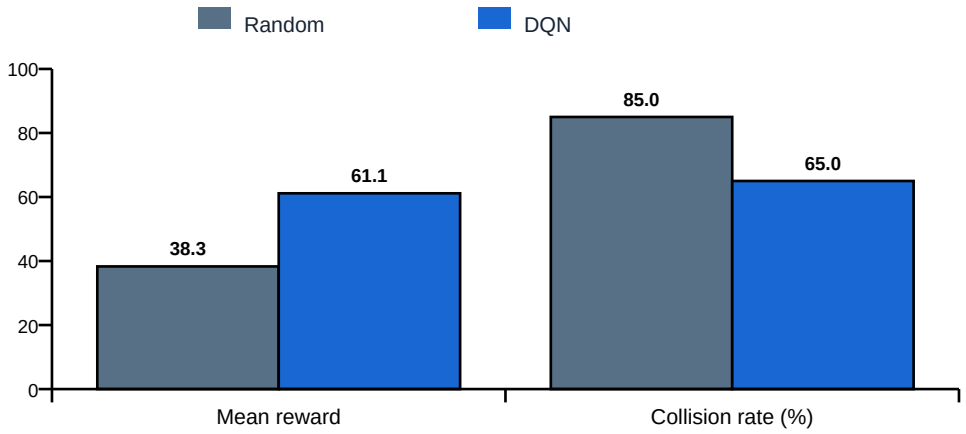
Simulation metrics show repeatable behavior under the documented configuration. They do not establish safety, real-world transfer, or statistically complete performance across all environment conditions.

3. Demo 1 - HighwayEnv DQN

A DQN maps the kinematic observation to five discrete high-level driving actions. Experiences are stored in a replay buffer and sampled for temporal-difference learning. A target network stabilizes the bootstrapped value estimate. Training and deterministic inference use the PyTorch ROCm device.

Training configuration

Parameter	Value	Parameter	Value
Timesteps	20,000	Network	256 x 256 MLP
Learning rate	5e-4	Replay buffer	15,000
Batch size	64	Gamma	0.8
Exploration	1.0 to 0.05	Device	PyTorch ROCm



Metric	Random	DQN	Difference
Mean total reward	38.31	61.15	+22.84
Mean episode steps	47.30	64.35	+17.05
Collision rate	85%	65%	-20 pp
Mean action inference	N/A	0.927 ms	N/A

The learned policy improves reward and episode duration while reducing collisions. The remaining 65% collision rate is deliberately reported: this is an entry-level training baseline rather than a mature driving system.

4. Demo 2 - Genesis Franka Target Following

A red target follows a deterministic three-dimensional path. Genesis position-level inverse kinematics computes a full Franka configuration for a downward-facing hand pose. The first seven joint targets are sent to a PD controller and four physics steps are executed per rendered frame.

Control sequence

Stage	Operation	Output
1	Generate fruit position from sinusoidal phase	3D target position
2	Offset target by 8 cm in height	Desired hand pose
3	Solve Genesis inverse kinematics	Seven arm-joint targets
4	Apply joint position PD control	Simulated arm motion
5	Render and measure	Frame, tracking error, IK time

Submitted result

Metric	Value
Mean tracking error	7.446 mm
P95 tracking error	10.750 mm
Maximum tracking error	10.990 mm
Mean IK time	2.243 ms
P95 IK time	2.341 ms
300-frame execution time	6.861 s

Accuracy note

This is dynamic target following, not physical grasping. The fruit is moved directly, gravity and collision are disabled, and the gripper does not close from contact feedback. The method uses position-level inverse kinematics and should not be described as differential IK.

5. AMD Radeon GPU and ROCm Utilization

The project uses AMD GPU computation in several stages. HighwayEnv DQN training and action inference run through PyTorch ROCm. Genesis uses its AMD GPU backend for simulator tensor work, inverse kinematics, physics execution, and off-screen rendering.

Evidence field	Submitted value
GPU	AMD Radeon Graphics
PyTorch	2.9.1+rocm7.2.1.gitff65f5bc
HIP	7.2.53211-e1a6bc5663
Genesis	1.3.1
Genesis backend record	3 (runtime enum string)

Runtime evidence commands

```
rocm-smi
amd-smi version
python -c "import torch; print(torch.__version__, torch.version.hip,
torch.cuda.get_device_name(0))"
```

PyTorch retains the `torch.cuda` API on ROCm for ecosystem compatibility. In this environment it addresses the HIP/ROCm device, not an NVIDIA CUDA installation.

Container reproducibility

The Dockerfile starts from AMD's published `rocm/pytorch:rocm7.2.1_ubuntu24.04_py3.12_pytorch_release_2.9.1` image and installs pinned direct project dependencies. GPU devices are passed at runtime through `/dev/kfd` and `/dev/dri`.

6. Contributions, Limitations, and Deliverables

Key contributions

- A dual-domain package spanning reinforcement learning and robot control on one AMD ROCm platform.

- Machine-readable GPU and framework evidence embedded in result artifacts.

- Seeded baseline comparison so the driving video is not the only learning evidence.

- Precise scope language that distinguishes following from grasping and position IK from differential IK.

- A headless workflow, pinned container, exact commands, expected outputs, and known limitations.

Limitations and future work

- Improve driving safety with reward redesign, longer training, PPO or stronger DQN variants, and broader evaluation.

- Enable Genesis gravity, collision, contact, gripper control, and vision-based target estimation.

- Add domain randomization and evaluate Sim-to-Real transfer on physical hardware.

- Repeat performance tests across GPU models and report confidence intervals.

Deliverables

Deliverable	Repository location
Source code	demo1_highway_dqn/ and demo2_franka_genesis/
Reproduction guide	README.md
Container	Dockerfile and requirements-lock.txt
Metrics	Both demo results/ directories
Architecture	docs/system_architecture.svg
Demonstration clips	assets/videos/
Technical report	output/pdf/PhysicalAI_ROCm_Technical_Report.pdf

Team contributions

This is an individual submission by **daotianyi**. The participant implemented the simulation setup, training and control code, AMD ROCm integration, evaluation, visualization, technical report, demonstration videos, and reproducibility package.

7. Reproduction Checklist and References

Evaluator checklist

- Confirm a supported AMD Radeon GPU and ROCm kernel driver.
- Build the Docker image or activate a ROCm PyTorch environment.
- Run the README GPU verification snippet.
- Train HighwayEnv DQN for 20,000 steps and evaluate seeds 100-119.
- Run the Genesis 300-frame target-following command with seed 42.
- Compare generated JSON outputs with committed results.
- Review videos together with the stated limitations.

References

1. AMD ROCm Documentation. Install PyTorch for ROCm on Radeon and Ryzen.
2. Towers et al. Gymnasium: A Standard Interface for Reinforcement Learning Environments.
3. Leurent. HighwayEnv: An Environment for Autonomous Driving Decision-Making.
4. Raffin et al. Stable-Baselines3: Reliable Reinforcement Learning Implementations.
5. Genesis Embodied AI. Genesis physics and robotics simulation platform.

End of report